

Managing Screen Flows Overview

Modes

The screen flows functionality makes it possible to work in two modes: client and server.

Server Mode

In this mode, the screen flows functionality was introduced. Most of its logic is executed in the server, and the result of this execution is sent to the browser. Any interaction of the user with a screen (for example, clicking a submit button) results in a screen reload and a new call to the server for the result of this action.

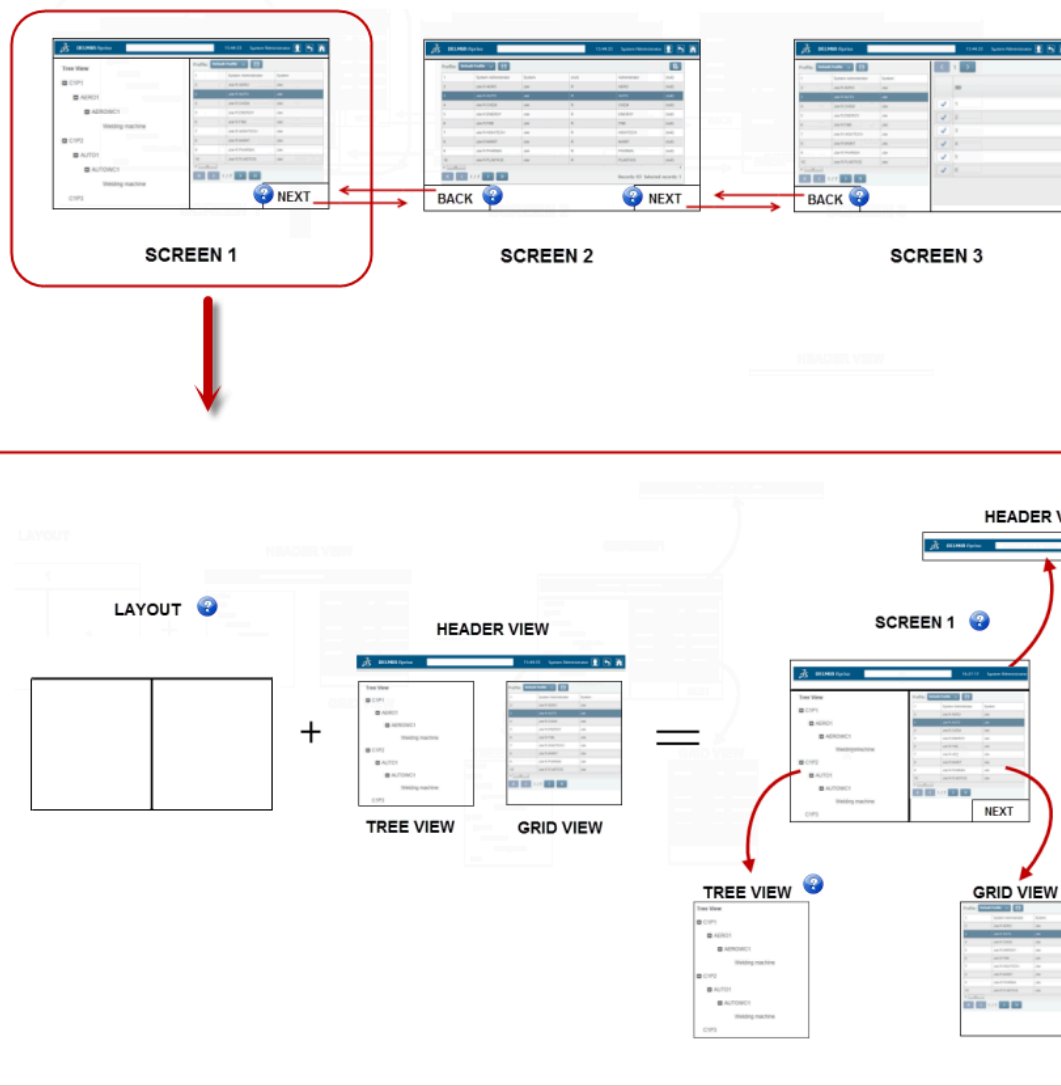
Client Mode

Due to the fast pace of changes in technology, new frameworks and technologies are appearing almost daily. Customer expectations regarding the UI, UX, and system responsiveness are also growing. To satisfy these needs, a new client-side mode is available in the screen flows functionality in the DELMIA Apriso Portal. The client-side mode helps to resolve technical issues such as connection loss, network traffic size, the number of requests on a single page, and the number of concurrent users on a web server.

For details, see [Screen Flows Client Mode](#).

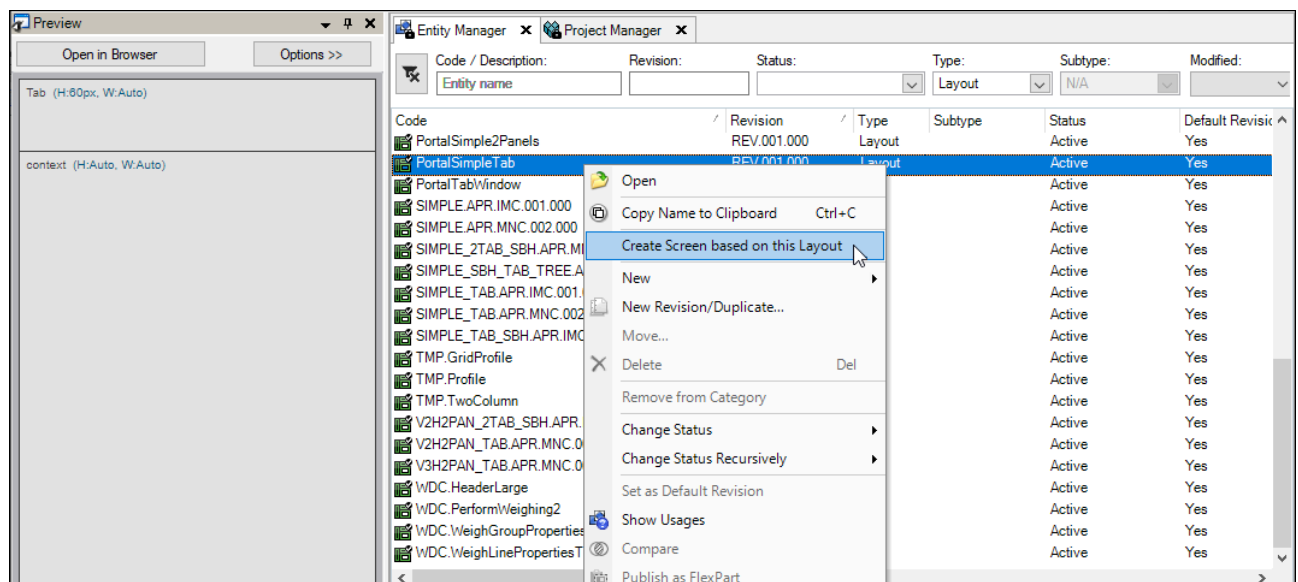
Concept of Designing Screen Flows

Click  on the picture for more details.

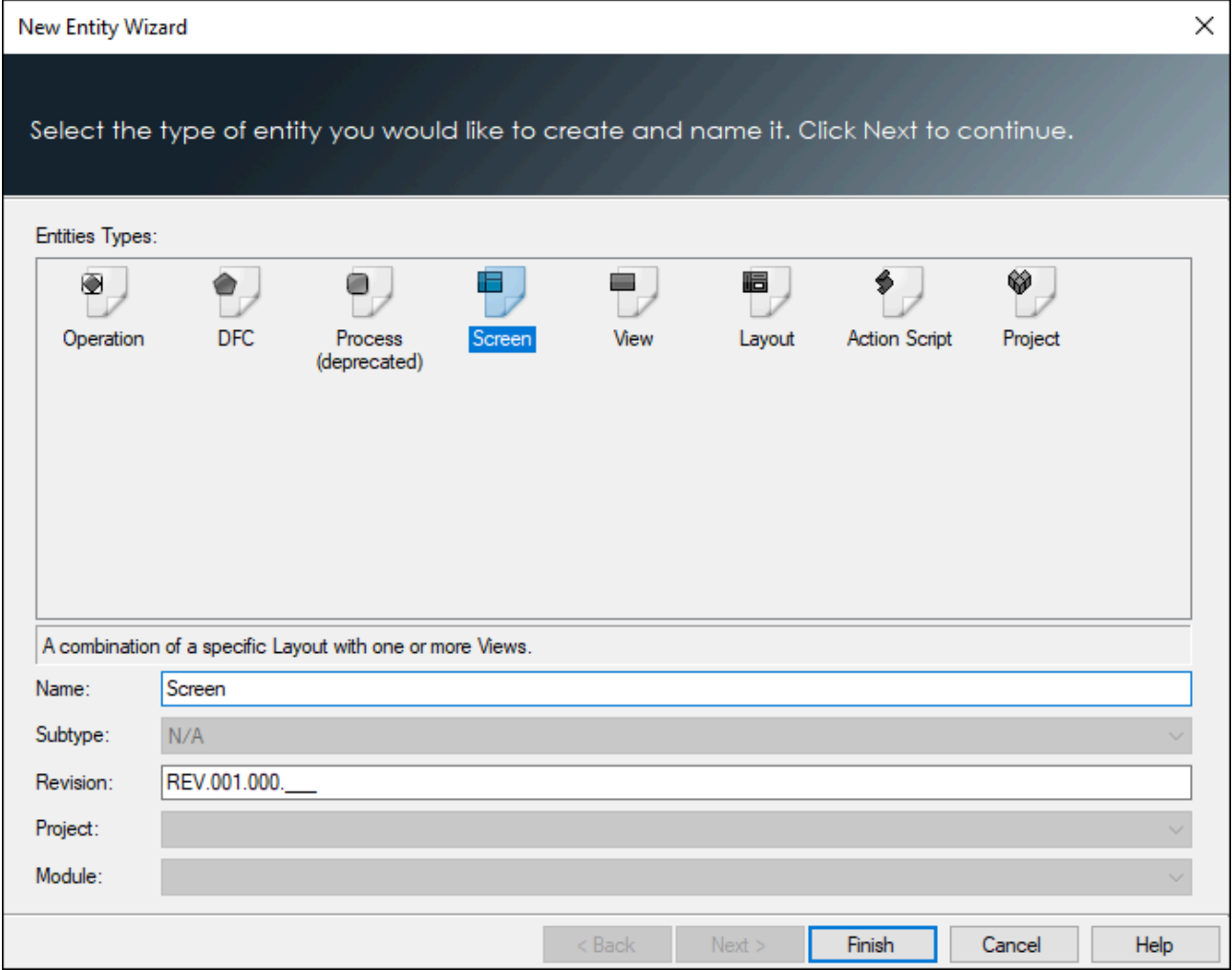


Screen Creation

1. Use the **Entity Manager** or **Project Entity Manager** to browse the existing layouts (use the **Type** filter).
2. Right-click a Layout and choose **Create Screen based on this Layout**.

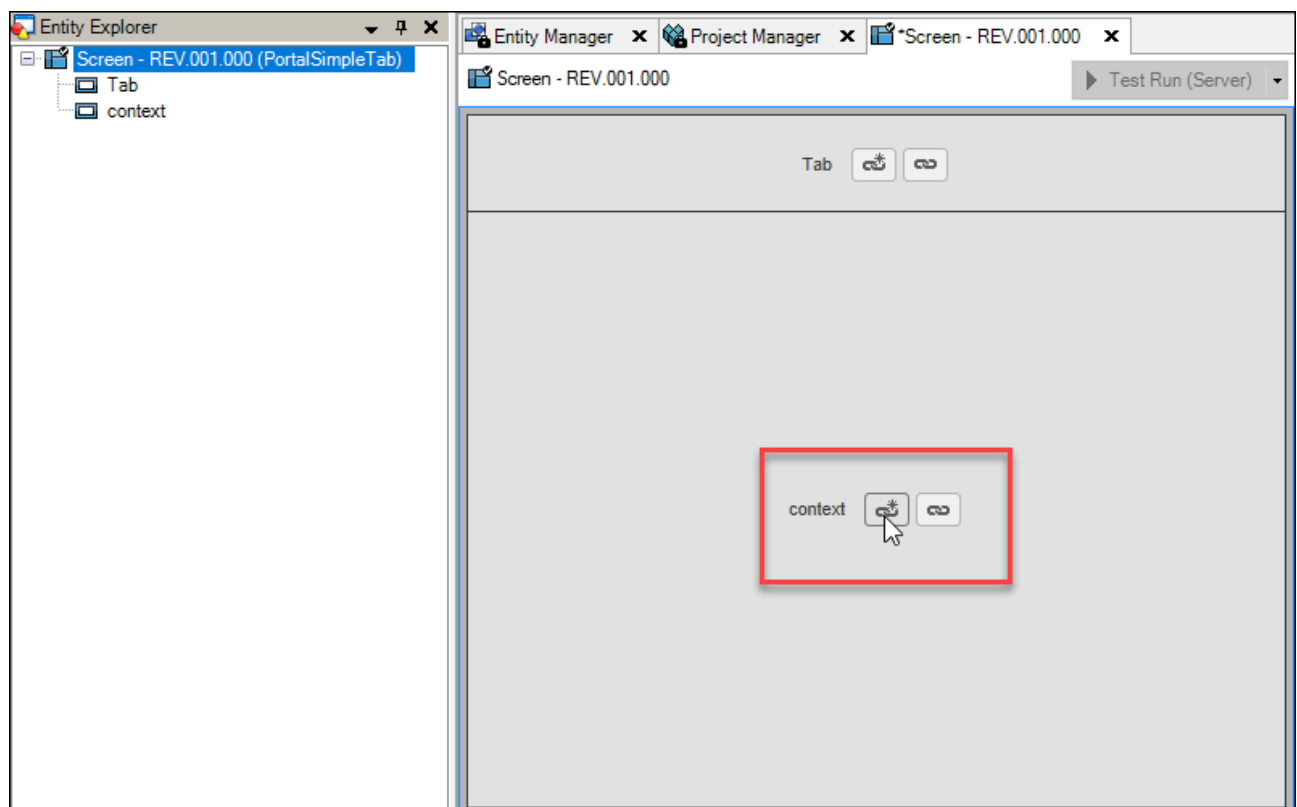


3. **New Entity Wizard** appears. Enter the screen's name and click **Finish**. The name cannot begin with a number.

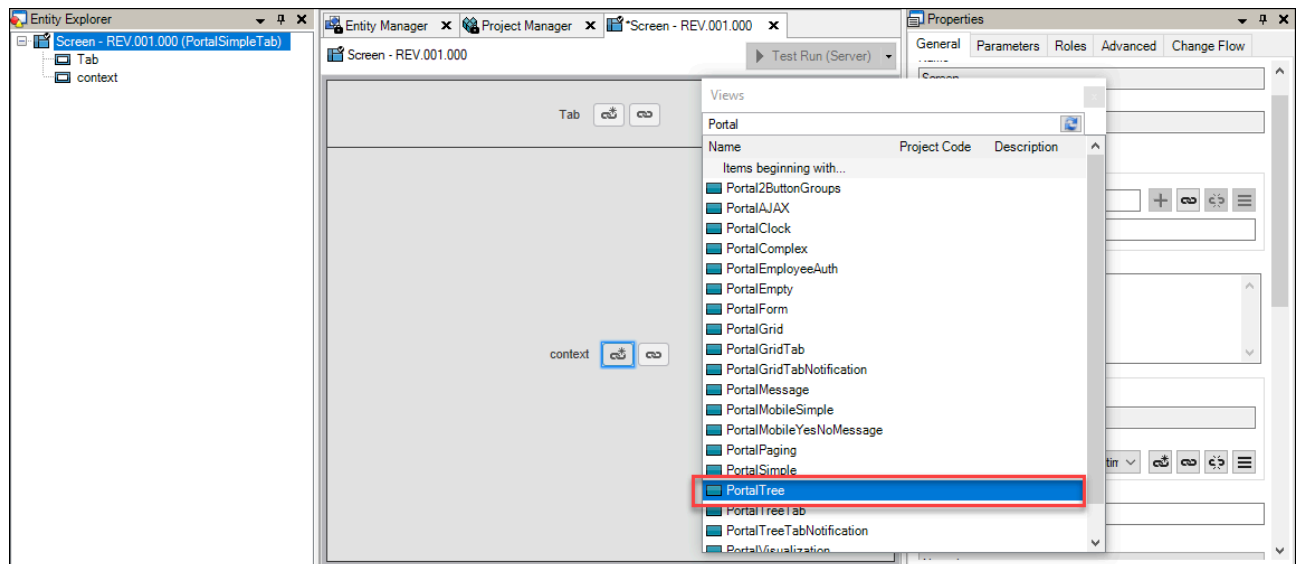


The **New Entity Wizard** dialog box is shown. It has a title bar with a close button. The main area has a dark header with the text: "Select the type of entity you would like to create and name it. Click Next to continue." Below this, under "Entities Types:", there are eight icons representing different entity types: Operation, DFC, Process (deprecated), **Screen** (highlighted with a blue border), View, Layout, Action Script, and Project. Below the icons, there is a text box that says "A combination of a specific Layout with one or more Views." Below this, there are several input fields: "Name:" with the value "Screen", "Subtype:" with the value "N/A", "Revision:" with the value "REV.001.000.____", "Project:", and "Module:". At the bottom, there are five buttons: "< Back", "Next >", **Finish** (highlighted with a blue border), "Cancel", and "Help".

4. Link a view entity to the layout using **Copy and Link View**. Using one of the existing templates is recommended.



5. Pick the view from the templates:



6. Duplicate the existing view.

Duplicate Entity [X]

Duplication options

☐ New entity revision

☒ New entity

New entity details

Entity name:

Entity revision:

☒ Set as default revision

Project details

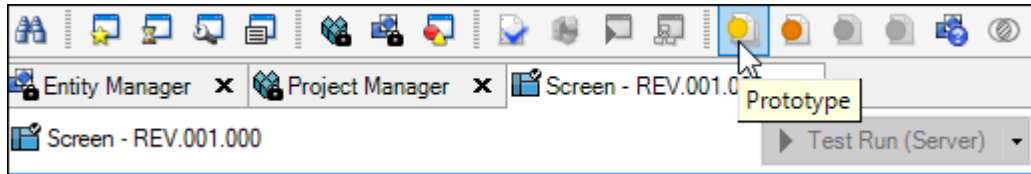
Project revision:

Project module:

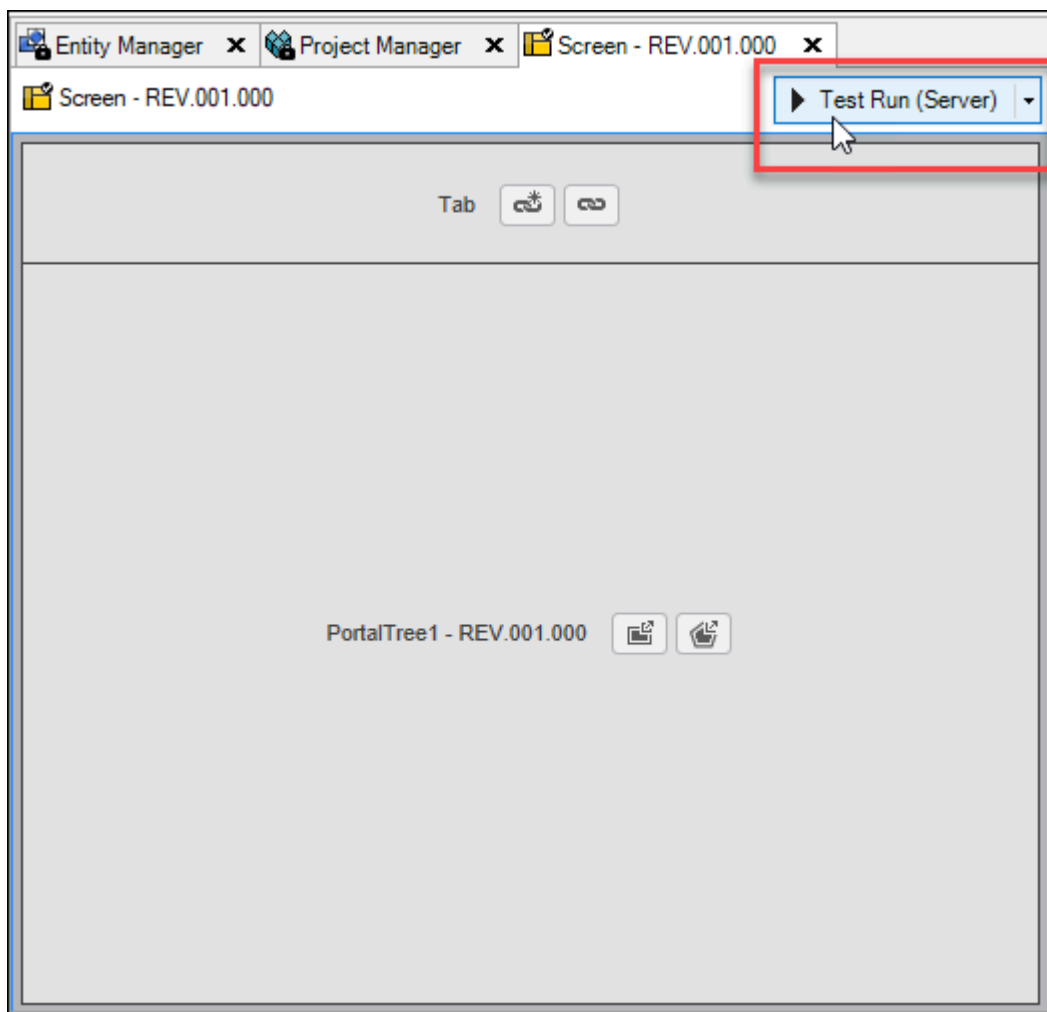
When using the **Copy And Link View** functionality, the copy of the view and its linked DFC is created automatically when a view and view DFC have the same name.

7. Make sure that all the created entities (layout, screen, views, DFCs) are in at least the **prototype** status.

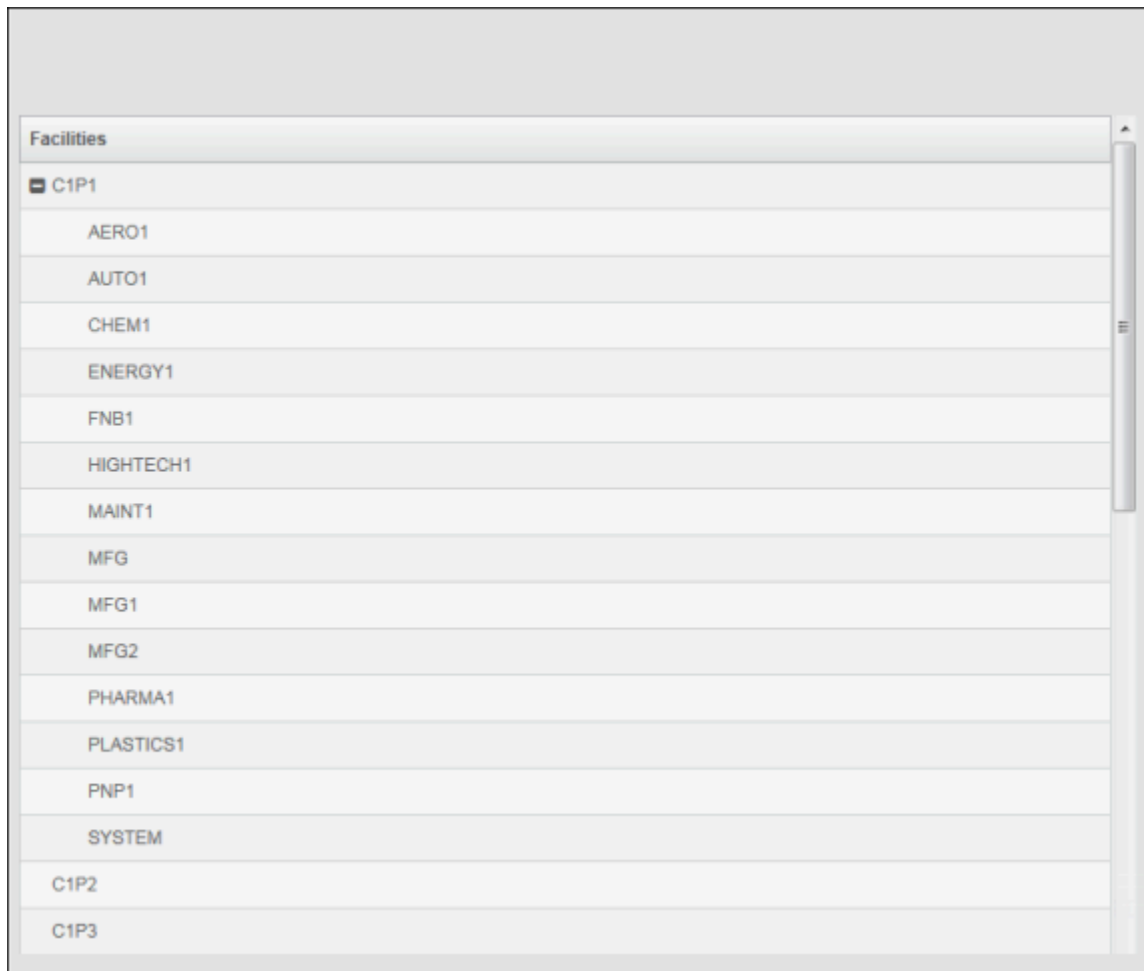
Be aware that changes made in the configuration of the screens, views, or actions will be automatically reflected only in Test Run. Otherwise, the old values will be taken from the cache, and you will need to wait for their expiration. The time after the cache is expired is controlled by the `ScreenFlowCacheExpirationSpan` key located in the `FunctionInterpreter` section of the Central Configuration file (for details, see [Central Configuration Documentation](#)).



8. Perform the **Test Run**.



The runtime view looks like this:



Business Process Implementation

Any business process that requires interaction with the end-user can be described with:

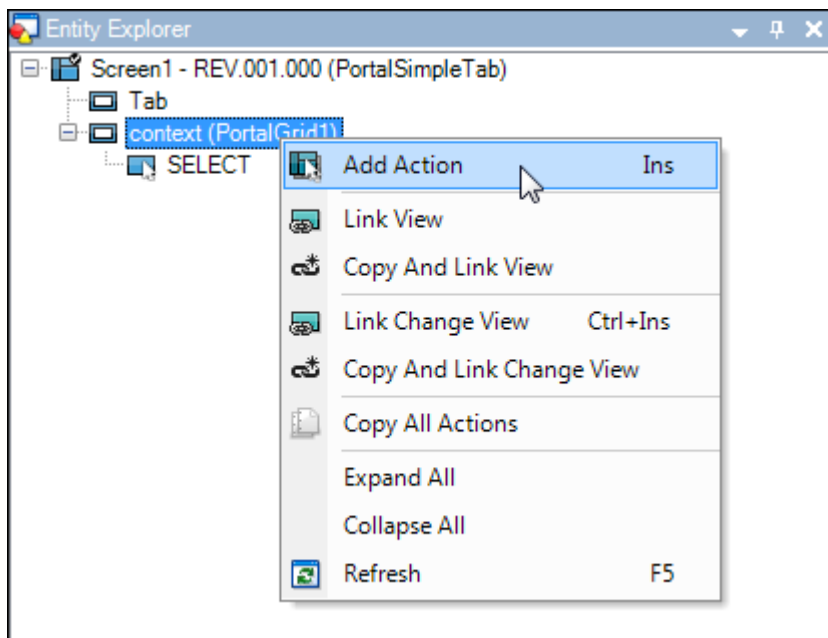
- ▶ **Screens** – definitions of what information should be displayed as well as how the information should be displayed to and taken from the user.
- ▶ **Actions** – definitions of possible events that can happen in the UI (either via user actions or based on external events).

A business process implemented using screens strongly separates the above areas. This simplifies the development and maintainability of the solution. It also allows the Process Implementer to directly link the user requirements specified in a detailed design with their technical implementation.

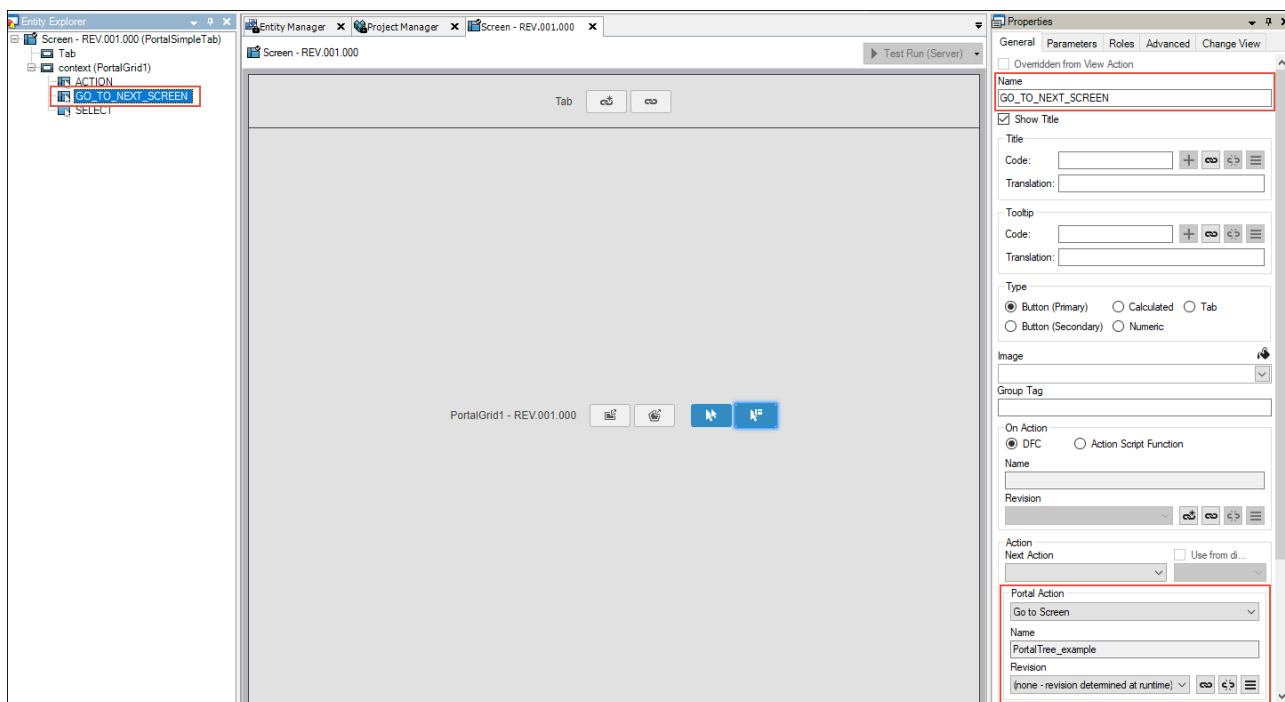
A solution implemented in this way consists of many simple, well-separated DFCs that execute basic functions rather than big and complex DFCs that mix business and UI logic.

Navigation to Another Screen

1. Open the screen created in the [previous](#) section.
2. Add an action. Use the **Entity Explorer** right-click menu:



3. Change an action name and link to a screen to which you want to navigate.



Screens, Layouts, and Views

Displaying up-to-date UI in DELMIA Apriso requires the use of several sub-DFCs, such as header, context, and footer. Dividing the UI per sub-DFC is a good practice because it allows for reusing the parts that are common between the different screens like the header or footer).

This approach is the basic concept for screen flows. Each screen in a screen flow consists of:

- ▶ A list of **views** that will be displayed in the screen.
- ▶ **Layouts**: definitions of the positions and sizes of the **panels** on which views will be linked.

Definitions of views, layouts, and screens are the only elements responsible for how the end-user UI is displayed.

Be aware that the Layout that is displayed may be the [embedded version of a Layout](#) (for details, see [Screen: General tab](#)).

Interaction Definition

As soon as you have defined what the UI will look like, the next step is defining the possible interactions of the end-user with the system using this UI. Such interactions are represented by **actions** and may be defined for every view. Each action defines a cause and effect:

- ▶ Cause – which UI element causes the event (for example, the click of a button or a selection change in the grid).
- ▶ Effect – the reaction to the action, which may be:
 - ▷ Execution of a DFC with business logic (for example, data validation, a database transaction).
 - ▷ Navigation to another screen (or a navigation command going back to a previous screen or exiting).
- ▶ Information exchange.

The screens and the business logic must exchange information. The **portal session** concept is the placeholder for all the variables exchanged between the views and the business logic DFCs.

In addition, managing screens keeps the screen call stack, which is used to support advanced navigation scenarios (for example, going back to the previous screen). Each stack element keeps the screen information and a portal session snapshot, which can be used later for rolling back to the previous session state.

Additional Information

In addition to UI-related elements, the screen flows feature also introduces a new methodology of designing and implementing business processes in DELMIA Apriso.

Managing screens is accomplished via multiple best practices for process and UI design, such as:

- ▶ Separation of the UI and business logic implementation.
- ▶ Adequate granularity of component implementation (which enables component reuse).
- ▶ Exposure of non-technical configuration to business users.
- ▶ An event-based approach to UI (that is, a more natural way of event handling for complex UI).

Modifying the Specific Properties of a Screen or View in the Active Status

Editing the specific properties of the active screens or views (ordinarily, active entities are uneditable) is available for:

- ▶ View actions (see [View Workspace: Actions Tab](#) and [View Action: General Tab](#)).
- ▶ View actions marked with  that are created on the screen (see [Define Actions](#)) or linked to the screen, but only when you use the **Override Action** option before changing the screen status to active.

To enable this option:

1. Open the CentralConfiguration.xml file located under <drive>\Program Files\Dassault Systemes\DELMIA Apriso 2025\Website\CentralConfiguration.
2. Find the ScreenFlowEditInActiveRevision key in the FunctionInterpreter section. For details, refer to [Central Configuration Documentation](#).
3. Set the key value to True.

```
<add key="ScreenFlowEditInActiveRevision" value="True" />
```

When the key is set, the properties that you can edit include:

- ▶ Changing the order of actions (see [View Workspace: Actions Tab](#))
- ▶ Enabling/disabling actions (see [View Workspace: Actions Tab](#)).
- ▶ Changing the image to be displayed on the UI element (see the **Image** setting on [View Action: General Tab](#)).
- ▶ Changing the descriptive action name that will be displayed on the button or tab control (see the **Title** setting on [View Action: General Tab](#)).
- ▶ Changing the text of the action tooltip that will be displayed after hovering the button with the mouse pointer (see **Tooltip** setting on [View Action: General Tab](#)).
- ▶ Changing the access to the action (see [View Action: Roles Tab](#))
- ▶ Switching the view action type between **Button (Primary)** and **Button (Secondary)** types (see the **Type** setting on [View Action: General Tab](#)).